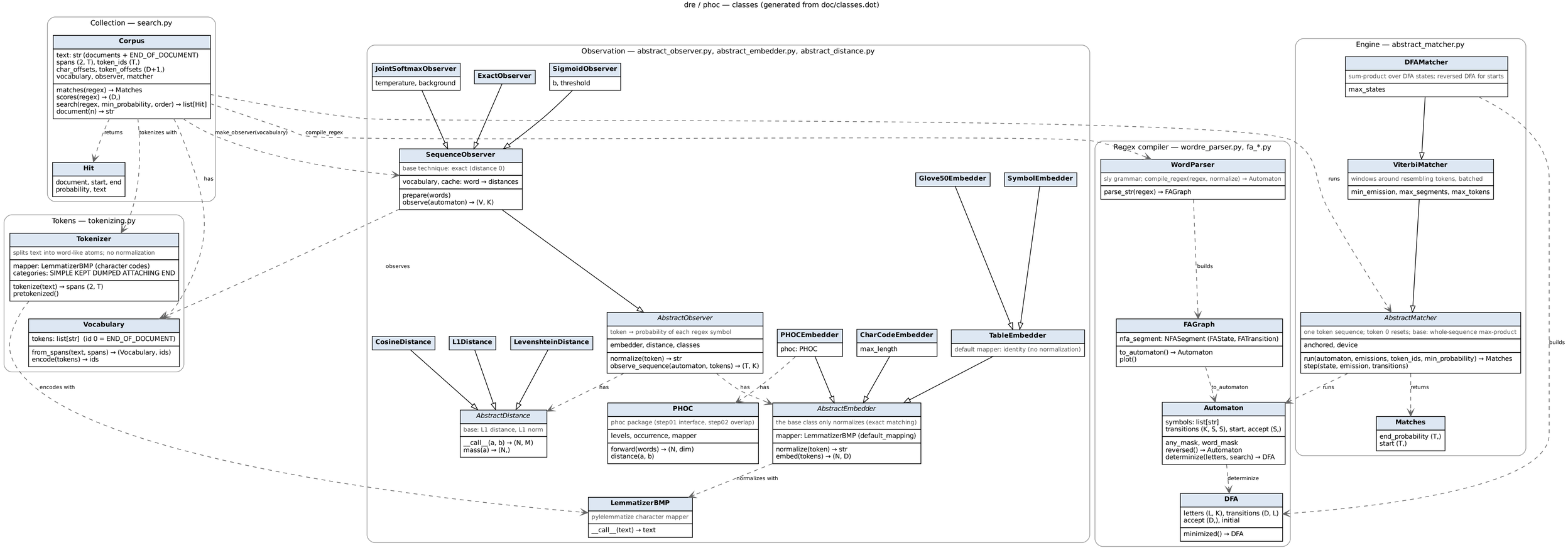
dre / phoc — classes (generated from doc/classes.dot)
Collection — search.py
Corpus
text: str (documents + END_OF_DOCUMENT)
spans (2, T), token_ids (T,)
char_offsets, token_offsets (D+1,)
vocabulary, observer, matcher
matches(regex) → Matches
scores(regex) → (D,)
search(regex, min_probability, order) → list[Hit]
document(n) → str
Hit
document, start, end
probability, text
returns
tokenizes with
has
Tokens — tokenizing.py
Tokenizer
splits text into word-like atoms; no normalization
mapper: LemmatizerBMP (character codes)
categories: SIMPLE KEPT DUMPED ATTACHING END
tokenize(text) → spans (2, T)
pretokenized()
Vocabulary
tokens: list[str] (id 0 = END_OF_DOCUMENT)
from_spans(text, spans) → (Vocabulary, ids)
encode(tokens) → ids
encodes with
make_observer(vocabulary)
compile_regex
Observation — abstract_observer.py, abstract_embedder.py, abstract_distance.py
JointSoftmaxObserver
temperature, background
ExactObserver
SigmoidObserver
b, threshold
SequenceObserver
base technique: exact (distance 0)
vocabulary, cache: word → distances
prepare(words)
observe(automaton) → (V, K)
observes
Glove50Embedder
SymbolEmbedder
AbstractObserver
token → probability of each regex symbol
embedder, distance, classes
normalize(token) → str
observe_sequence(automaton, tokens) → (T, K)
CosineDistance
L1Distance
LevenshteinDistance
PHOCEmbedder
phoc: PHOC
CharCodeEmbedder
max_length
TableEmbedder
default mapper: identity (no normalization)
has
AbstractDistance
base: L1 distance, L1 norm
__call__(a, b) → (N, M)
mass(a) → (N,)
PHOC
phoc package (step01 interface, step02 overlap)
levels, occurrence, mapper
forward(words) → (N, dim)
distance(a, b)
AbstractEmbedder
the base class only normalizes (exact matching)
mapper: LemmatizerBMP (default_mapping)
normalize(token) → str
embed(tokens) → (N, D)
normalizes with
LemmatizerBMP
pylelemmatize character mapper
__call__(text) → text
Regex compiler — wordre_parser.py, fa_*.py
WordParser
sly grammar; compile_regex(regex, normalize) → Automaton
parse_str(regex) → FAGraph
builds
FAGraph
nfa_segment: NFASegment (FAState, FATransition)
to_automaton() → Automaton
plot()
to_automaton
Automaton
symbols: list[str]
transitions (K, S, S), start, accept (S,)
any_mask, word_mask
reversed() → Automaton
determinize(letters, search) → DFA
determinize
DFA
letters (L, K), transitions (D, L)
accept (D,), initial
minimized() → DFA
Engine — abstract_matcher.py
DFAMatcher
sum-product over DFA states; reversed DFA for starts
max_states
runs
ViterbiMatcher
windows around resembling tokens, batched
min_emission, max_segments, max_tokens
AbstractMatcher
one token sequence; token 0 resets; base: whole-sequence max-product
anchored, device
run(automaton, emissions, token_ids, min_probability) → Matches
step(state, emission, transitions)
runs
returns
Matches
end_probability (T,)
start (T,)
builds